



GIREF – Université Laval

Méthode SSFEM : Des Fondements à l'Implémentation Haute Dimension

Cartographie algorithmique et Feuille de route

El Mehdi EN-NAHAS

M1 Mathématiques Appliquées (IP Paris / ENSTA)

Stage de Recherche - Été 2026

Table des matières

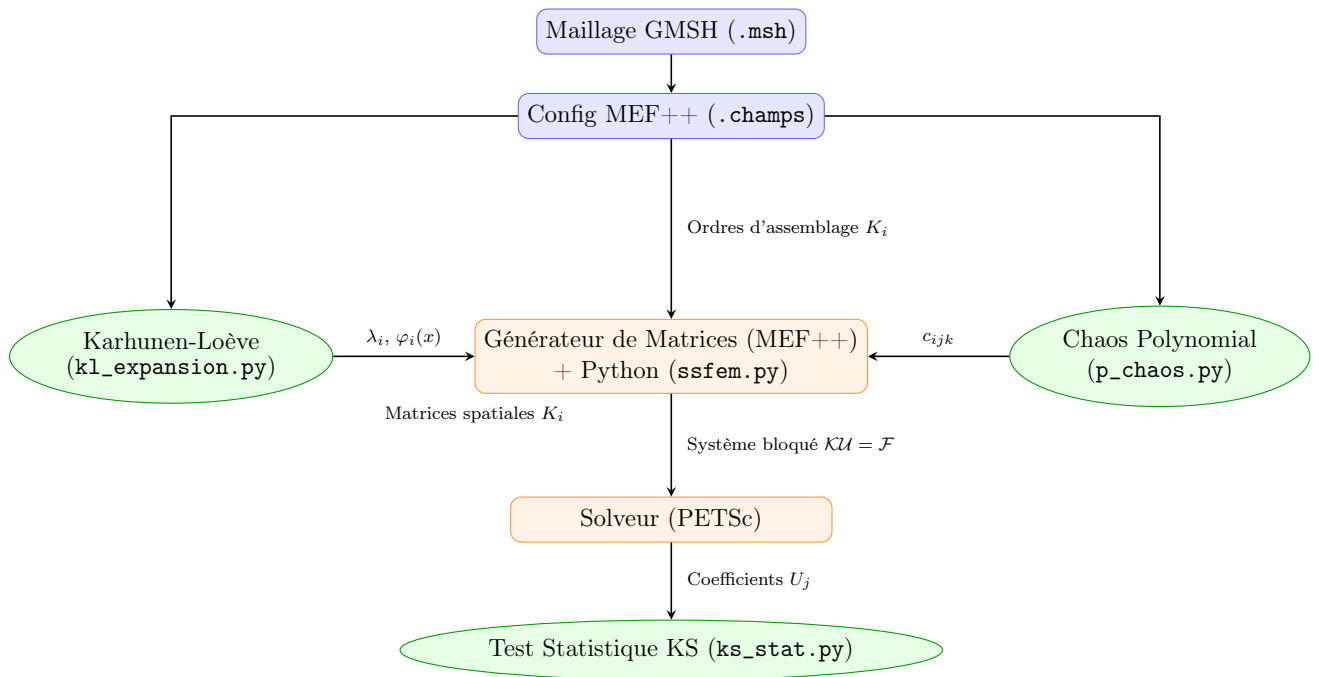
1	Carte Mentale : Architecture du Projet SSFEM	2
2	L'Entrée : Karhunen-Loève et le défi de la Haute Dimension	3
2.1	Le problème de la dimensionnalité (1D vs 2D/3D)	3
2.2	La solution (Ton Projet) : Random Fourier Features (RFF)	3
3	La Sortie : Chaos Polynomial (PC) et l'Algorithme Combinatoire	4
3.1	La combinatoire des multi-indices (L'algorithme des Balles et Boîtes) . . .	4
3.2	Le calcul du tenseur c_{ijk}	4
4	L'Assemblage de la Matrice Globale (ssfem.py)	5
5	Validation Statistique : Le test de Kolmogorov-Smirnov	5
6	Feuille de Route et Prochaines Étapes	6

Objectif du document

Ce document constitue le "Point Zéro" du projet. Il détaille l'architecture logicielle existante (MEF++ et Python), les fondements de la méthode Stochastic Spectral Finite Element Method (SSFEM), les verrous mathématiques du passage en haute dimension (Théorème de Bochner), ainsi que les algorithmes combinatoires sous-jacents au Chaos Polynomial.

1 Carte Mentale : Architecture du Projet SSFEM

L'objectif de la SSFEM est de transformer un problème mécanique à paramètres aléatoires en un système d'équations déterministe massif, résolu en une seule passe. Voici la cartographie de l'implémentation.



Le flux de travail détaillé :

1. Le fichier `ssfem.py` charge la configuration physique via le fichier `.champs`.
2. Il utilise `kl_expansion.py` pour discrétiser l'incertitude du matériau en M variables.
3. Il demande à MEF++ de calculer les matrices de rigidité spatiales K_i pour chaque mode de KL.
4. Il utilise `p_chaos.py` pour calculer les tenseurs de probabilité c_{ijk} .
5. Il assemble la Matrice Globale $K = \sum c_{ijk} K_i$ et laisse PETSc inverser le système pour trouver la solution probabiliste exacte.

2 L'Entrée : Karhunen-Loève et le défi de la Haute Dimension

L'expansion de Karhunen-Loève (KL) permet de réduire la dimensionnalité d'un champ aléatoire continu en une somme de variables décorréées :

$$E(x, \theta) = \mu(x) + \sum_{i=1}^M \sqrt{\lambda_i} \varphi_i(x) \xi_i(\theta) \quad (1)$$

2.1 Le problème de la dimensionnalité (1D vs 2D/3D)

Pour trouver les couples propres (λ_i, φ_i) , il faut résoudre l'équation de Fredholm :

$$\int_{\Omega} K(x, y) \varphi(y) dy = \lambda \varphi(x) \quad (2)$$

où $K(x, y)$ est le noyau de covariance (ex : le noyau gaussien $K(x, y) = \exp(-\|x - y\|^2/L^2)$).

- **En 1D** : Sur un segment simple, les solutions de cette intégrale sont connues analytiquement. C'est ce qui est codé dans le fichier actuel `kl_varphi.py`.
- **En 2D/3D** : Sur un maillage complexe (le cylindre), l'équation de Fredholm n'a pas de solution analytique fermée. Résoudre ce problème aux valeurs propres numériquement sur une matrice de taille $N \times N$ (où $N = 10^5$ nœuds) est prohibitif. **C'est ici que s'arrêtait le stage précédent.**

2.2 La solution (Ton Projet) : Random Fourier Features (RFF)

Pour contourner la résolution de Fredholm en haute dimension, nous utiliserons le Théorème de Bochner. Un noyau continu défini positif stationnaire peut être représenté comme la transformée de Fourier d'une mesure de probabilité $p(\omega)$.

Pour un noyau gaussien, $p(\omega)$ est également une loi gaussienne. On peut donc approcher la covariance mathématiquement par une espérance :

$$K(x, y) \approx \frac{1}{D} \sum_{i=1}^D Z_{\omega_i}(x) \cdot Z_{\omega_i}(y) \quad (3)$$

Où le vecteur de caractéristiques $Z(x)$ est défini (comme vu sur le tableau blanc du laboratoire) par :

$$Z(x) = \frac{1}{\sqrt{D}} \begin{bmatrix} \cos(\omega_1 \cdot x) \\ \sin(\omega_1 \cdot x) \\ \vdots \\ \cos(\omega_D \cdot x) \\ \sin(\omega_D \cdot x) \end{bmatrix} \quad (4)$$

Pourquoi c'est révolutionnaire pour le code ? Nous n'avons plus besoin de résoudre de grosses matrices pour trouver les $\varphi_i(x)$. Il nous suffira de tirer des fréquences ω aléatoirement, et d'évaluer de simples cosinus et sinus sur les coordonnées (x, y, z) des nœuds du maillage MEF++.

3 La Sortie : Chaos Polynomial (PC) et l'Algorithme Combinatoire

Puisque les paramètres d'entrée sont incertains, la solution (le déplacement U) s'écrit dans une base de polynômes orthogonaux (Chaos Polynomial) :

$$U(\theta) = \sum_{j=0}^{P-1} U_j \Psi_j(\boldsymbol{\xi}) \quad (5)$$

3.1 La combinatoire des multi-indices (L'algorithme des Balles et Boîtes)

Si nous avons M variables aléatoires $\boldsymbol{\xi} = (\xi_1, \dots, \xi_M)$, et que nous cherchons des polynômes de degré total inférieur ou égal à q , nous devons trouver tous les multi-indices $\boldsymbol{\alpha} = (\alpha_1, \dots, \alpha_M)$ tels que :

$$\alpha_1 + \alpha_2 + \dots + \alpha_M \leq q \quad (6)$$

C'est le rôle exclusif de la méthode codée dans `p_chaos.py`. L'algorithme utilise la technique *Stars and Bars* (Balles et Séparateurs) :

- On dispose de q boîtes vides (représentant le degré disponible).
- On dispose de $M - 1$ balles (représentant les séparateurs entre les variables).
- **Le fonctionnement** : L'algorithme déplace les balles de la gauche vers la droite pour générer toutes les répartitions possibles. Le nombre de boîtes vides avant la première balle donne la puissance α_1 , entre la 1ère et la 2ème balle donne α_2 , etc.
- Le nombre total de termes de notre base polynomiale sera donc $P = \frac{(M+q)!}{M!q!}$.

3.2 Le calcul du tenseur c_{ijk}

L'équation résiduelle de la méthode de Galerkin impose de calculer la projection de l'équation sur notre base polynomiale. Cela fait apparaître le tenseur déterministe suivant :

$$c_{ijk} = \mathbb{E} [\xi_i \Psi_j(\boldsymbol{\xi}) \Psi_k(\boldsymbol{\xi})] \quad (7)$$

Puisque les variables ξ sont indépendantes et que les polynômes Ψ sont orthogonaux (Polynômes d'Hermite pour une loi normale), la majorité des c_{ijk} sont nuls. Le script `p_chaos.py` calcule analytiquement ce tenseur (un tenseur creux) qui servira de "colle" pour assembler la matrice globale.

4 L'Assemblage de la Matrice Globale (ssfem.py)

C'est ici que l'espace (MEF++) et la probabilité (Python) fusionnent. L'équation stochastique discrétisée de notre système prend la forme :

$$\sum_{i=0}^M \sum_{j=0}^{P-1} c_{ijk} K_i U_j = F_k \quad \forall k \in \{0, \dots, P-1\} \quad (8)$$

Dans le script `ssfem.py`, l'assemblage se fait via la définition de blocs matriciels K_{jk} :

$$K_{jk} = \sum_{i=0}^M c_{ijk} K_i \quad (9)$$

Où K_i sont les matrices spatiales (de dimension $N_{ddl} \times N_{ddl}$) générées par MEF++. La matrice envoyée à PETSc via la commande `PETSc.Mat().createNest()` est donc une matrice par blocs gigantesque de dimension $(P \times N_{ddl}) \times (P \times N_{ddl})$:

$$\mathcal{K} = \begin{bmatrix} K_{0,0} & K_{0,1} & \dots & K_{0,P-1} \\ K_{1,0} & K_{1,1} & \dots & K_{1,P-1} \\ \vdots & \vdots & \ddots & \vdots \\ K_{P-1,0} & K_{P-1,1} & \dots & K_{P-1,P-1} \end{bmatrix} \quad (10)$$

La résolution de ce système unique donne directement accès au vecteur global $\mathcal{U}$, contenant les P blocs U_j , décrivant intégralement la solution stochastique.

5 Validation Statistique : Le test de Kolmogorov-Smirnov

Le fichier `ks_stat.py` permet de valider la solution analytique SSFEM.

- **Objectif** : Prouver que l'échantillonnage de notre Chaos Polynomial (qui est une somme de polynômes) reproduit correctement la loi de probabilité physique réelle (obtenue par exemple par de vrais tirages Monte Carlo).
- **Principe du KS-Test** : Le test compare la Fonction de Répartition Empirique (CDF) de notre solution SSFEM échantillonnée avec la CDF de référence.
- **Métrique** : Il calcule la distance de Kolmogorov $D = \sup_x |F_{SSFEM}(x) - F_{Ref}(x)|$. Si la *p-value* associée est supérieure au seuil usuel (0.05), on ne peut pas rejeter l'hypothèse que les deux distributions sont identiques, validant ainsi notre implémentation mathématique.

6 Feuille de Route et Prochaines Étapes

Pour atteindre les objectifs du stage de recherche, voici la séquence d'actions à mener à partir de ce document de base :

1. **Création de l'environnement de Test (1ère semaine)**
 - S'assurer de la bonne exécution d'un cas simple de Monte Carlo avec le script `mc.py` dans le dossier de travail `STAGE`.
 - Étudier la syntaxe du fichier `cylindre_cis.champs` pour s'habituer aux "Termes Faibles" et aux pré-post traitements MEF++.
2. **Codage de la méthode Random Fourier Features (Python)**
 - Créer un script Python (ex : `rff_kernel.py`) qui génère des fréquences ω_i selon une loi gaussienne.
 - Coder la fonction $Z(x)$ générant les termes en cosinus et sinus pour un vecteur spatial (x, y, z) donné.
 - Valider visuellement (avec matplotlib) que $E[Z(x) \cdot Z(y)]$ approche bien le profil d'une cloche gaussienne 1D, puis 2D.
3. **Interface avec MEF++ (Le pont critique)**
 - Remplacer l'injection statique des φ_i (boucle de lecture des chaînes de caractères de `kl_varphi.py`) dans `ssfem.py`.
 - Créer une méthode permettant d'évaluer le vecteur $Z(x)$ pour chaque nœud du maillage MEF++, et assigner ces valeurs à un champ MEF++ pour que la matrice spatiale K_i puisse être assemblée.
4. **Assemblage Haute Dimension et Benchmarks**
 - Tester l'assemblage de la matrice géante $\mathcal{K}$ sur un maillage de cylindre 3D.
 - Valider les statistiques de la solution (via `ks_stat.py`) par rapport à une résolution Monte Carlo "force brute".